

# Reviewer Pack — Noncommutative $L^p$ Norm Lower Bounds for Disjointness Commun...

Entry d090c650622d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 18:03:31 UTC

## Noncommutative $L^p$ Norm Lower Bounds for Disjointness Communication Matrices

**Entry ID:** d090c650622d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 18:03:31 UTC

### 1. Conjecture

**Field A** (mathematical branch): Noncommutative  $L^p$  Geometry **Field B** (complexity object): Communication Complexity of DISJOINTNESS

#### Statement:

For any  $n \times n$  matrix  $M$  representing a DISJOINTNESS instance, the noncommutative  $L^p$  norm  $\|M\|_p \geq \Omega(\sqrt{n})$  for  $p \in (1, 2)$ . This holds for all matrices where the support is a product set  $S \times T$  with  $|S| = |T| = n^{1/2}$ .

#### Rationale (proposer's reasoning):

Noncommutative  $L^p$  norms capture operator-theoretic structure of tensor products, which may encode geometric obstructions to efficient communication. The product support constraint ensures the matrix lies in the intersection of a secant variety and a noncommutative convex body, creating a natural test case for norm-communication tradeoffs.

**Taxonomy category:** COMM\_DISJ (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 8073626a8c61db3f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.95       | UNCERTAIN        | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    n = 40
    p = 1.5
    threshold = 0.1 * math.sqrt(n)

    random.seed(seed)
    S = set(range(20))
    T = set(range(20))
    M = [[random.choice([0, 1]) if i in S and j in T else 0 for j in range(n)] for i in range(n)]

    # Compute the noncommutative L^p norm
    M_scaled = [[M[i][j] / (n * n) ** (1/p) for j in range(n)] for i in range(n)]
    norm_p = 0.0
    for _ in range(500):
        v = [random.random() for _ in range(n)]
        v_norm = sum(x**p for x in v) ** (1/p)
        u = [M_scaled[i][j] * v[j] for j in range(n)]
        u_norm = sum(x**p for x in u) ** (1/p)
        norm_p += u_norm / v_norm

    norm_p /= 500.0

    result = {
        "metric_name": "noncommutative_Lp_norm",
        "metric_value": norm_p,
        "instances_tested": 1,
        "conjecture_holds": norm_p >= threshold,
        "counterexample": "" if norm_p >= threshold else f"Matrix M did not meet the norm bound (n"
    }

    return result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_norm_p = sum(r["metric_value"] for r in results) / len(results)
std_norm_p = math.sqrt(sum((r["metric_value"] - mean_norm_p) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_norm_p} std={std_norm_p} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Matrix M did not meet the norm bound\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d597251a.py", line 56, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d597251a.py", line 34, in run_trial
    u = [M_scaled[i][j] * v[j] for j in range(n)]
          ^

```

UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed due to UnboundLocalError, preventing data collection. Pre-registered support condition cannot be evaluated without valid results. | next: Fix the code bug in test\_d597251a.py line 34 (unbound 'i' variable) and re-run experiments

## 11. Audit log (LLM calls)

**Total LLM calls:** 7

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 39130        |
| 2 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 12803        |
| 3 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 9950         |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 8846         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14172        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 7803         |
| 7 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 17366        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110071 ms total latency. Provider mix: {'ollama\_remote': 7}

(full prompt+response transcripts available in *research/audit/d090c650622d.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/d090c650622d.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/d090c650622d.tar.gz* (if generated)